🔥 PalletTracker v2.2 — Guía de Testing Firebase

✅ Lo que cambió en v2.2

Componente	Cambio
`build.gradle` (raíz)	Agregado `google-services:4.4.0`
`build.gradle` (app)	Firebase Database + Auth + Analytics activos
`google-services.json`	Copiado del proyecto Aunload (compartido)
`FirebaseManager.java`	TODOs reemplazados por código real de Firebase SDK
`AjustesFragment.java`	Test de conexión asíncrono, URL auto-configurada
`AndroidManifest.xml`	Permisos `INTERNET` + `ACCESS_NETWORK_STATE`
Versión	`2.2` / versionCode `7`

📱 Preparación (2+ dispositivos)

Dispositivo 1 (“Scanner-1”)

1. Instalar `PalletTracker-2.2-debug.apk`
2. Abrir → Menú → **Ajustes**
3. Sección **Sincronización**:
   - Activar switch de sincronización → **ON**
   - Nombre del dispositivo: `Scanner-1`
   - Toca **Probar Conexión** → debe decir ✅ Conexión exitosa

Dispositivo 2 (“Scanner-2”)

1. Repetir los mismos pasos
2. Nombre: `Scanner-2`
3. Probar conexión → ✅

🧪 Test 1: Crear Sesión

En **Scanner-1**:
1. Desde la pantalla principal, **doble-tap en el FAB** (botón flotante)

2. Ingresar cantidad esperada: `10`
3. El FAB debe mostrar `0/10`

Verificar en Firebase Console:
- Ir a `https://console.firebase.google.com`
- Proyecto: `aunload-rescue`
- Realtime Database → `/sessions/{sessionId}/`
- Debe existir un nodo con `expectedPallets: 10`

🧪 Test 2: Escanear Pallet en Scanner-1

1. Activar servicio de accesibilidad (Ajustes → Accesibilidad)
2. Abrir la app del escáner (ej. Honeywell)
3. Escanear un pallet
4. El FAB debe cambiar a `1/10`

Verificar en Firebase:
- `/sessions/{id}/pallets/` debe tener el pallet
- `/sessions/{id}/scannedPallets` = `1`
- `/sessions/{id}/activeDevices/{deviceId}/palletsScannedByDevice` = `1`

🧪 Test 3: Sincronización Multi-Dispositivo

En **Scanner-2**:
1. Unirse a la misma sesión (el código de sesión se comparte)
2. Escanear otro pallet diferente

Verificar en ambos dispositivos:
- Scanner-1 debe recibir notificación: "📦 Scanner-2 escaneó: {palletCode}"
- FAB en ambos debe mostrar `2/10`
- Firebase debe tener 2 pallets + 2 dispositivos activos

🧪 Test 4: Manejo de Duplicados

Intentar escanear el **mismo pallet** en ambos dispositivos:
- Solo debe registrarse **una vez** (key = palletCode)
- El segundo intento debe ser ignorado silenciosamente

🧪 Test 5: Container Completo

Escanear hasta completar los 10 pallets entre ambos dispositivos:
- FAB debe cambiar a **verde** (#4CAF50) con animación de pulso
- Ambos dispositivos deben ver el estado verde

🧪 Test 6: Estado de Dispositivos

En **Ajustes → Sincronización**:
- "📱 Dispositivos activos: 2" (cuando ambos están conectados)
- Si un escáner cierra la app por >30 seg → aparece como offline

---

🧪 Test 7: Reconexión

1. En Scanner-1, cerrar la app completamente
2. Reabrir PalletTracker
3. Debe reconectarse automáticamente a la sesión guardada
4. Los pallets de Scanner-2 deben aparecer

⚠️ Notas Importantes

- **Firebase URL**: Auto-configurada desde `google-services.json` (proyecto `aunload-rescue`)
- **No requiere login**: Realtime Database con reglas abiertas para desarrollo
- **Heartbeat**: Cada 10 segundos cada dispositivo reporta su estado
- **Conflictos**: Timestamp como árbitro — el más reciente gana
- **Almacenamiento**: Dual write (local SharedPrefs + Firebase)

🔐 Seguridad (Para Producción)

Antes de usar en producción, configurar reglas en Firebase Console:

```
{
  "rules": {
    "sessions": {
      "$sessionId": {
        ".read": true,
        ".write": true
      }
    },
    "connection_test": {
      ".read": true,
      ".write": true
    }
  }
}
```

Y eventualmente agregar autenticación Firebase Auth.